

SF3 Machine Learning

Project Information

Easter Term 2026

Last updated: April 24, 2026

Logistics

Key dates

- There is an introductory session at 9:15 am on Friday 15 May in the James Building Dyson Seminar Room
- The interim report is due on Friday 29 May at 4pm (submitted via Moodle)
- The final report is due on Friday 12 June at 4pm (submitted via Moodle)

Demonstration sessions

These will take place in the James Dyson Building Seminar Room on Fridays (9:15-10:45am and 2:00-3:30pm) and in LR3A on Tuesdays (11:00-12:30).

Some important points:

- Session attendance is not compulsory, **except** you must attend a session on the week after submitting your interim report to receive feedback. A calendar sign-up link will be posted on Moodle to sign up for feedback.
- On the last week of the project, there will be extra sessions on Monday 08 June available over Zoom to help you prepare for writing your final report
- On Friday 05 June, sessions will be held over Zoom instead of in person (lecture rooms are in use for exams)
- On Tuesday 09 June, the session will be in the James Dyson Building Seminar Room (LR3A will be in use for exams on that day, please do not go there)

The final week of the project will be extremely busy. It is **strongly advised** that you use the extra Monday sessions and the Tuesday session of the final week to ask any last-minute questions you have about the project or the report. Between the final demonstration session and your submission deadline, demonstrators and project leaders (e.g. if contacted via email or Moodle) are unlikely to be able to respond quickly enough for you to incorporate any of their answers to your question in your report. Plan ahead!

Reports

While write-ups of scientific experiments typically use a top-level structure that follows “Introduction, Methods, Results, Discussion”, technical reports describe ongoing or completed technical work. This means that they can follow a structure that is closer to chronological, where the method you used can immediately precede the results and discussion it generated, followed by the next method and the next result. The sections below provide guidance on approximately how much space you should dedicate to each topic in your reports.

Interim report (20 marks)

The interim report should not exceed 3 A4 pages, nor should it exceed 750 words (including figure captions and section headings). This report should describe your progress over the first two weeks of the project, and it is therefore expected that you have finished the nonlinear modelling component of the project and are preparing to move on to control of the system.

While both reports should be professionally presented (i.e. free of typographical errors, typeset neatly, legible if printed on A4 paper, and generally coherent), it is understood that the interim report reflects work that is currently in progress. For example, this means that figures that are rougher than we would expect in a final report are acceptable in the interim report (and you should use your feedback session to discuss how their presentation can be improved ahead of your final report). Similarly, the discussion is not expected to be as detailed or deep as in the final report.

This report should include the following content in roughly the following ratios:

- 15% a very brief overview of the project
- 70% a description of your progress to date (including results, technical issues encountered and how they were resolved, and commentary on how this might affect future work)
- 15% a plan describing how you intend to proceed with the rest of the project

Final report (60 marks)

The final report should not exceed 11 A4 pages, nor should it exceed 3000 words (including figure captions and section headings, but not including any references). This report should be a description of your full project (i.e. it should also describe the work you wrote about in the interim report, although it is likely you will want to condense or re-write those sections in light of your later results). Any appendices (code, or the sample prompts detailed in the AI use policy document) should be uploaded to Moodle as separate files.

The word and page limits mean that you will need to be deliberate about how you present information: there is a lot of content to pack into a small space, but this should not be done at the expense of clarity. The week-by-week project outline provides suggestions of several types of plots you can use to present information, but you will not have space to supply every single one of these plots for every situation. Think carefully about how you will get your message across.

The final report should include:

- An introduction to the project, including a high-level summary (approx. 1 page)

- A technical description of the work undertaken in the project and its results, broken out into sensible subsections (e.g. splitting out modelling, control, and extensions, with each of these sections combining to make 7-8 pages total)
- If including, a critical evaluation of AI use during the project (approx. 1 page)
- A discussion, including recommendations for improvement and critical evaluation of your work (approx. 2 pages)

AI Use Policy

This project has a dedicated document available separately on Moodle describing an AI use policy. Students are expected to be familiar with this policy (which encourages certain ways of using AI tools but forbids others). Students using AI tools in the ways encouraged by that document are expected to provide a critical evaluation of their effectiveness in their Final Report, with further details available in the AI Use Policy document.

General advice

Backing up and versioning your work

You should, from the very start of the project, make sure that you are sensibly versioning and backing-up your work. It is your responsibility to make sure that you do not lose your work (e.g. in the case of unexpected hardware failure or loss). `git` is a good choice, especially if you are familiar with it (and if not, demonstrators can help introduce you), but the final choice of versioning/back-up is left to your personal preference.

Keeping a record of your work

You are expected to maintain an ongoing record of your work. You can do this digitally or in a physical lab notebook. Record (with dates) what you are working on, notes, scrap working, plots, and any other items that will help you keep track of the project. These notes will be very helpful to refer back to when writing your reports, and demonstrators will ask to see them when discussing your project.

Week 1: Simulation and simple linear models

Getting set up

The Moodle page for SF3 contains a `cartpole.py` file, which has code to help you simulate the dynamics of the inverted pendulum. You are welcome to use department computers, your own personal computer, or online coding environments such as Deepnote Cloud or Google Colab for this project (although the latter category may be relatively slow for model fitting). Equally, you are welcome to choose for yourself whether to use Jupyter notebooks or standalone Python scripts for this project.

Use the code below to verify that you can import from `cartpole.py`. If you see a (very rudimentary) animation of the system, then you are set up¹.

```
from cartpole import CartPole

# visual=True turns on animation (don't use this in other sections!)
example_system = CartPole(visual=True)

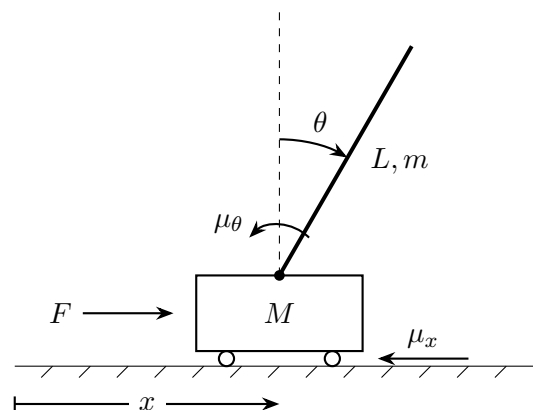
cart_position = 0.0
cart_velocity = 0.1
pole_angle = 0.01
pole_velocity = 0.0

state = [cart_position, cart_velocity, pole_angle, pole_velocity]

example_system.setState(state)

for _ in range(100):
    example_system.performAction()
```

Dynamical simulation



Consider the inverted pendulum system (“cartpole”) drawn above with a freely rotating pendulum attached to a freely moving cart. The state of the system is described by the following variables: position and velocity of the cart $x, \dot{x}$, angle and angular velocity of the pole $\theta, \dot{\theta}$, with the angle being periodic on $[-\pi, \pi]$. The centre position of the cart corresponds to $x = 0$, and

¹The animation will play if running this as a standalone script, but it may only produce a static image of the system if run in certain notebook environments. This is fine, as the animation is for illustrative purposes only.

the pole hanging vertically down corresponds to $\theta = \pi$. This system moves under the influence of gravity and an external action force.

The equations of motion of the system are:

$$3\ddot{x} \cos \theta + 2L\ddot{\theta} = 3g \sin \theta - 6\mu_\theta \dot{\theta} / mL \quad (1)$$

$$(m + M)\ddot{x} + \frac{1}{2}mL\ddot{\theta} \cos \theta - \frac{1}{2}mL\dot{\theta}^2 \sin \theta = F - \mu_x \dot{x} \quad (2)$$

Where F is the external action (force) on the cart, μ_x and μ_θ are the friction coefficients of the cart and the pole, respectively. These equations will be familiar to you if you have attended the 3F2 lab, and you will even have derived them yourself if you did the 3F2 FTR.

Task 1.1

Study the code in the `cartpole.py` file, which creates a Python class to describe the system. Note that the variables describe the state of the system and the `performAction()` function updates the state variables using the Euler algorithm (applying several steps at each action) using a given force on the cart (which is the ‘action’). Passing in a value of zero for the force corresponds to the free dynamics of the system.

Write code to simulate a ‘rollout’. This is a run with a specified initial condition simulated for a number of time steps. You can achieve this by using the `performAction()` function in a loop with no applied force, starting from the stable equilibrium position and some nonzero initial cart velocity or angular velocity. Plot the resulting time evolution of the system variables, varying the size of the initial velocities to realise different behaviours, such as a simple oscillation around the stable equilibrium or a complete rotation of the pendulum. You can plot all variables as a function of time, and you can also plot pairs of variables against one another (similar to phase portraits).

Hint 1: Useful ranges are as follows. Cart velocity: $[-10, 10]$, pole angle: $[-\pi, \pi]$, pole (angular) velocity: $[-15, 15]$.

Hint 2: Note how the angle is used in the dynamics as a continuous variable, rather than just in the range $[-\pi, \pi]$. There is a `remap_angle` function in the `CartPole` module that you can use to get the angle in the usual range. *This will be an important consideration later on when we develop models of the dynamics.*

Changes of state

The inverted pendulum is a special case of a system that we can handle with a simple linear controller: we know where the stationary points are, and we have access to equations of motion that can be linearised about those points. However, in general, we do not know the equations of motion that govern the evolution of a physical system, but we do have *observations* of how the system behaves. In these cases, we have to take a different approach: to use our observations to develop a *model* for the time evolution of the system.

The model is a function $f(X)$ that takes the current state of the system, and maps it onto a new state, which is its prediction for the state at a later time. Let the state of the system be described by a vector X , given by

$$X = [x, \dot{x}, \theta, \dot{\theta}] \quad (3)$$

Given the current state X , let us call Y the state of the system after a single call to the `performAction()` function (with no force applied).

Task 1.2

To investigate the functional relationship between X and Y , initialise the system using a random value for all state variables, and then scan through one of the state variables in a suitable range, plotting Y as a function of your scan.

Hint: In this case, unlike rollouts, you should reset all the state variables after each call to `performAction()`.

You should observe that the relationship between X and Y as defined above is nearly linear, which is not surprising because the change after one step is small. This is something we can use to our advantage: instead of taking the new state vector itself as the target of our model, we can instead model the *change* in the state vector. We can define the new target for modelling as $Y \equiv X(T) - X(0)$, where $X(t)$ represents the time evolution of the state under the dynamics, and T corresponds to a single call to `performAction()`.

Explore this new functional relationship again (i) using scans of single variables, and (ii) using contour plots where you take slices of the data in two of the variables while keeping the other two fixed. One of the variables has no effect on the next step—which one?

Hint: The `tricontourf` function of `matplotlib` is very useful for contour plots.

Note: in principle, we could model arbitrary time shifts instead of a single call to `performAction()`, but the longer the time shift, the more complex the model needs to be.

A simple linear model

The simplest model is a linear one, where the target Y is assumed to be a linear function of the current state X , i.e.:

$$f(X) = CX \tag{4}$$

where C is a 4×4 matrix of coefficients.

Task 1.3

Gather data in the form of pairs of state vectors (X, Y) by initialising the simulator in a random state (picking suitable ranges) and running it for one step. Start with 500 data points and use a linear regression on your dataset to find the optimal coefficient matrix C , then test your predictions against the data.

Hint: One way to plot the results is to put the input state variable on the horizontal axis and to put both the predicted state variable and the real next step on the vertical axis. Another way is to put the target data (i.e. the next real step) on the horizontal axis and the predicted next step on the vertical axis. In this latter plot, a perfect prediction would correspond to a perfect straight line.

You should repeat the scans from the previous task, simultaneously plotting the real change in state with your predicted change in state as a function of your scan. Which variables are predicted well by the linear model and which are not? Why?

Task 1.4

The true test of the model is how well it can predict the time evolution of the physical system. Iterate the model to predict the time evolution, using various initial conditions to compare how

accurate the predictions are with respect to the true dynamics when initialised from the same initial conditions (Note: the model here is deterministic, with no noise added).

Since your models above predict the *change* in the state variable, the iterated time evolution is:

$$X_{n+1} \leftarrow X_n + f(X_n) \quad (5)$$

where X represents the state of the system at step n .

Make plots that illustrate the comparison between the true dynamics and your model's predictions, including ones where the pole makes a full circle.

If your angle θ leaves the angle range without remapping, your solution with the iterated model will diverge. Why is that the case? Ensure that you remap the angle during your iterations. Do the true dynamics of the system use explicit remapping? Why not?

Week 2: Nonlinear modelling

Kernel functions

You should have found in the previous week that the linear model does not do a particularly good job of capturing the true system. In order to improve our ability to model the system, we will need to resort to nonlinear techniques.

As a next step, you are going to implement a nonlinear model by using linear regression with nonlinear basis functions. For simplicity, we will start by constructing a model of a single state variable (for example, of the cart's position). Given a dataset of (X, Y) pairs, the model function is given by:

$$f(X) = \sum_i \alpha_i K(X, X_i) \quad (6)$$

where the sum runs over the basis functions, where α_i is the coefficient for each term in the sum, and where K is a *kernel function* that is used to define a nonlinear basis. Noting for the single variable case, Y is scalar and corresponds to the next true value of the single variable we are interested in, but X is still a vector providing information about the entire current state of the system.

The kernel function takes two arguments. The first is X , the state vector where you evaluate the basis function, and the second is X_i , another state vector which we use to place the basis function somewhere in state space. To make the basis functions relevant, we take the set of locations $\{X_i\}$ to be a subset of the gathered data points.

We will start with a Gaussian kernel function,

$$K(X, X') = e^{-\sum_j \frac{(X^{(j)} - X'^{(j)})^2}{2\sigma_j^2}} \quad (7)$$

where the notation $X^{(j)}$ refers to the j^{th} component of the state vector (i.e. if we start counting from $j = 0$, then $X^{(0)}$ is the cart position x , $X^{(1)}$ is the cart velocity $\dot{x}$, etc.). The parameters σ_j are *length scale* hyperparameters of the model, and they need to be known, guessed or fitted.

Note 1: there is an important caveat for using this kernel function in our current situation. One of our state vector components, θ , is periodic. We can introduce this periodicity into our kernel function by substituting $\sin^2((\theta - \theta')/2)$ in place of $(\theta - \theta')^2$ in the part of the kernel function that corresponds to the angle variable. The factor of $1/2$ that ensures that the periodicity is 2π and the kernel value remains between zero and one.

Note 2: if we were to place a basis function at every location at which we have data and then use Equation 6, this would be equivalent the mean of a *Gaussian process* with covariance given by the kernel K . This framing of the regression problem is particularly helpful if the data is stochastic (e.g. it has some noise component that we can model).

Note 3: to extend the single variable model to our system, we simply create four separate models—one for each of the state variables.

Finding the parameters for the nonlinear model

Suppose that we have N data locations. We can rewrite the functional form for $f(X)$ in Equation 6 in terms of a matrix multiplication:

$$f(X) = K_{NN} \alpha_N \quad (8)$$

Where the subscript N is used to denote the size of the array (rather than an index) and K_{NN} is an $N \times N$ array with elements:

$$[K_{NN}]_{i,i'} = K(X_i, X_{i'}) \quad (9)$$

We can then set this equal to the target function values:

$$K_{NN}\alpha_N = Y_N \quad (10)$$

With Gaussian basis functions, the condition number of the matrix is very large, and a direct solution of the above linear system would be unstable. One way to get around this is to regularise. Tikhonov regularisation modifies this to:

$$(K_{NN} + \lambda I)\alpha_N = Y_N \quad (11)$$

with solution

$$\alpha_N = [K_{NN} + \lambda I]^{-1}Y_N \quad (12)$$

where λ is a parameter. The smaller the values of λ , the closer is the fit to the data, but the more unstable the linear system. Interestingly, this is also exactly the form of the mean solution in the Gaussian process regression problem of inference in the presence of noisy input data, with the identification that λ is the variance of the data noise.

Suppose now that while we collected N pairs of (X, Y) datapoints, we instead chose a subset of size M of the X locations to serve as basis function centres. The linear system is then no longer square:

$$K_{NM}\alpha_M = Y_N \quad (13)$$

where K_{MN} is an $M \times N$ matrix with elements corresponding to the M basis locations and all the N data point locations. Recalling our Part IB Linear Algebra material, we have fewer unknown coefficients than data points, and so the problem is over-determined. The least squares solution would be:

$$\alpha_M = [K_{MN}K_{NM}]^{-1}K_{MN}Y_N \quad (14)$$

The matrix enclosed in square brackets may be badly conditioned, so once again we may need to regularise. This can be accomplished as below (see Note 2 for an explanation):

$$\alpha_M^{(j)} = (K_{MN}K_{NM} + \lambda K_{MM})^{-1} K_{MN}Y_N^{(j)} \quad (15)$$

where K_{MM} is an $M \times M$ matrix with elements that are given by the kernel function evaluated between the locations.

When applying the above to the cartpole system (and fitting your four separate models, one for each component of the state vector), you will find that you do not yet have a method to guide the selection of λ or the length scale parameters σ_j . For now, experiment with a few different values for λ (e.g. between 10^{-6} and 10^{-1} , using a log scale) and see which values work better on validation data. A reasonable first guess for each value of σ_j is the standard deviation (in your dataset) for each corresponding state variable.

Note 1: you should *never* use the `np.linalg.inv` function to invert the matrix for solving a linear system of equations, because it can be numerically unstable for ill-conditioned matrices. A better alternative is to use `np.linalg.lstsq`, which directly solves the least-squares problem for equations of the form $A\mathbf{x} = \mathbf{b}$. This concept should be familiar to you from Part IB.

Note 2: Rather than simply adding a multiple of the identity matrix like the square case, we instead look to Gaussian process inference for a better idea. It turns out that a sparse Gaussian process corresponds to this case and, there, the model is written as a conditional probability not on the original N data values directly, but on the unknown function values at the M data locations that are chosen for the basis function centres. The interpretation of this is that the least squares system is Tikhonov regularised, but the regularisation is evaluated in the kernel-norm, not the Euclidean norm.

Task 2.1

Fit a model of the system using the data you gathered in Week 1. For your target function, you could choose either the change in state after one step (as before), or the error of your linear model in the change in state. Verify (by making plots) that the nonlinear model does indeed fit the data. Study the convergence of the model (i.e. the systematic reduction in error) as a function of increasing both the amount of data and the number of basis functions (e.g. start with $M = 10$ and increase by factors of 2, selecting the data locations for the basis randomly from the data).

You should also plot 2D slices of your target function and the fit, as well as using rollouts to illustrate how closely the iterated model matches the real dynamics of the system for a wide range of sensible initial conditions.

How long does your model show reasonable agreement with the real dynamics? Quantify this both in units of time and in terms of the number of oscillation cycles. How well does this compare to ‘point-wise’ accuracy measured from random test data (e.g. a randomly selected state)?

Gradient-based hyperparameter tuning

The JAX library

In the previous section, the hyperparameter values for λ and the length scale parameters σ are chosen by trial and error given a few candidate values. A better alternative is to tune these hyperparameters with gradient-based optimisation. In order to do this, we will rely on the `jax` Python library (<https://docs.jax.dev>).

This library uses automatic differentiation transformations to provide efficient evaluation of gradients. With `import jax.numpy as jnp`, we can replace most standard calls to `numpy` functions with their `jax` equivalents. For example, `np.tanh` becomes `jnp.tanh`, with the advantage that we can obtain a function that calculates the gradient of the operations done by our code via `jax.grad`. In this case, to calculate the gradient of the `tanh` function, we could simply use `jax.grad(jnp.tanh)`. This returns a new function that calculates the gradient.

Another useful advantage of JAX is that it includes built-in Just-In-Time (JIT) compilation. Using `jax.jit`, we can compile functions for faster execution. To build on the example above, `jax.jit(jax.grad(jnp.tanh))` returns a compiled function that evaluates the gradient of the `tanh` function much faster than with standard Python. However, the *first* time the function is called, it will be slow, as JAX has to compile the function; subsequent calls are very fast.

JAX has support for parallelising computations over CPU cores or a GPU. Finally, JAX functions can be automatically vectorised to efficiently map them over arrays representing batches of inputs. This avoids using Python’s native loops, which are slow. For example: if we have a Python function `test_func` that operates on scalars, we can obtain a corresponding vectorised

version of the function that operates on vectors by using `jax.vmap(test_func)`. In this case, when evaluated over a vector, `jax.vmap(test_func)` will return a new vector of the same shape as the input vector with each entry corresponding to the evaluation of `test_func` on each entry of the input vector.

Task 2.2

The objective of this task is to write code for fitting nonlinear models with JAX, doing gradient-based hyperparameter optimisation.

Write a function that takes as input the hyperparameter values, fits the model on the training data, and returns the performance metric (Note 1) of the fitted model on validation data. Obtain the gradient of that function using JAX and use an optimiser (Note 2). Report the resulting performance metrics on the validation before and after optimisation. What are your optimal length scale values? Why do you think you obtained those values? Verify, using plots, that the optimised model does indeed fit the data better.

Next, use rollouts to investigate how closely the optimised model matches the real dynamics for a wide range of sensible initial conditions. How long does your model show reasonable agreement with the real dynamics? Quantify this as in Task 2.1 and compare your results.

Note 1: A suitable choice here would be the Mean-Squared Error (MSE). If you are learning the variance of Gaussian noise variables, the negative log-likelihood is a more suitable choice.

Note 2: A good choice of optimiser would be the `scipy` library's `scipy.optimize.minimize` function. You should configure this to use the L-BFGS-B solver method and provide it with sensible initial values, as well as adding minimum and maximum thresholds for each parameter.

Improving feature representation

Despite tuning the hyperparameters, the previous models may *still* not be performing at a satisfactory level. One of the main reasons is that it is difficult to make predictions when we use the angle variable θ directly as an input, as its effect is invariant to adding multiples of 2π . While we tried to mitigate this earlier by using a modified kernel function with $\sin^2$, the $\sin$ function alone is not enough to fully identify an angle value. A better choice can be to use both sine and cosine of the angle as input features.

Task 2.3

Change your code to use both the sine and cosine of θ as input features for the nonlinear model (and remove the modification to the kernel that added the $\sin^2$), treating these features in the same way other features are treated in the model. At this point, you should be able to remove any calls to the angle remapping function. Refit both your linear and nonlinear models under this modification and comment on the results (you may find that making a plot to illustrate this point is helpful).

Week 3: Controlling the system

Control

Now that you have developed a good model to predict the dynamics of the system, it is time to control it. In this exercise, you will create a controller that achieves a desired state by parameterising the actions of the controller and then optimising those parameters through evaluation of the resulting performance. This is called *reinforcement learning* as a general term, and the specific case of using the interaction of the controller with the system to improve it is called *direct policy search*. There exist many other (more sophisticated) strategies.

When you call the `performAction(action)` function, it takes a signed scalar which is interpreted as an external force on the cart. The value you provide to the function is first passed through a `tanh` function before being interpreted as a force, as this prevents application of excessively large forces (this transformation is controlled by the `max_force` variable).

The first thing you will need to do is to modify your models, both linear and nonlinear, to take account of this new state variable: the dynamics of your system now have 5 variables (the 4 you have been using so far, and now the force parameter).

Task 3.1

Modify your code so that the input vector now includes the action taken. Collect new data, again using random initial conditions and one step, but this time include the action. Verify using the same tools as previously (scatter-plots, 1D and 2D scans, roll-outs) that your models can predict the change in the state variables. You will need to choose reasonable minimum and maximum values for the forces applied to remain predictable. What are the maximum and minimum forces that you can apply while your model still makes accurate predictions?

Policies

A *policy* is a function $p(X)$ (often written with π instead of p , but we want to avoid the use of the symbol π in a problem with a circular geometry to avoid ambiguity) that defines what action should be taken given the other state variables. The goal is to find a policy function that, when enacted, gives rise to the desired behaviour. In this case, we want to balance the pole around its unstable equilibrium position.

Loss functions

In order to optimise a policy, we need to mathematically define what we want to achieve. This is captured in a *loss function* (also called *objective function*), which is a measure of how close we are to the desired behaviour. In our case, we want the pole to be upright, so we could simply use the loss function:

$$l(X) = -\cos \theta \quad (16)$$

However, we can do better: if we want to keep the pole upright, we might also want to target having zero change in the system. So, more generally, we can define a *target state* X_0 which we want the system to achieve (e.g. $X_0 = [0, 0, 0, 0]$). We want a loss function that increases when the distance from the target is larger. For example, we could use:

$$l(X) = 1 - e^{-|X-X_0|^2/2\sigma_t^2} \quad (17)$$

where σ_l is a scaling factor (you could also introduce a separate one for each component of the state variable). The advantage of this form is that for large departures from the target, the loss is independent of the state. This expresses the notion that if the pole is far away from being upright and stationary, the loss function does not care what exactly it is doing.

The loss above is for a given state, but we wish to keep the pole upright continuously. So, over some time interval with T steps, the total loss of a trajectory is:

$$L = \sum_{t=1}^T l(X_t) \quad (18)$$

The `CartPole` class already contains a function to evaluate the pointwise loss $l(X)$.

Linear control

We start by defining a linear policy,

$$p(X) = \mathbf{p} \cdot \mathbf{X} \quad (19)$$

with unknown coefficients in the vector $\mathbf{p}$.

Task 3.2

Write code to evaluate the loss function for the trajectory of a rollout. Use a short time horizon, but long enough to capture a couple of oscillation periods.

Before any optimisation, visualise the loss function as you vary some parameters of $\mathbf{p}$, using similar 1D and 2D scans as in previous weeks.

Given a fixed initial state, optimise the unknowns in the policy to minimise the loss function under the evolution given by the true model dynamics. You can achieve this by using gradients given by JAX and by replacing calls to `numpy` in the `CartPole` class with their JAX equivalents. You can speed up your code by using `jax.jit` and by using `jax.scan` to replace `for` loops. As before, you can use `scipy.optimize` to minimise the loss function.

Start by finding elements of p that are able to stabilise the pole when it is initialised only slightly displaced from the upright position. Then, try it for the case where the pole is in its downward position.

To get good results for these experiments, you will need to tune the σ_l parameters in the loss, and you may also need to change `max_force`. Which values of σ_l did you choose? What happened when you started from the downward stable position?

Plot the time evolution of the variables under the policy to demonstrate that the pole is kept upright.

Task 3.3

In the previous task, you used the real dynamics of the pendulum to optimise your policy. Now, instead of using the real dynamics, try to optimise the policy parameters by testing them on model-rollouts using your good *model* of the dynamics from your work in Week 2. You will need to limit the time horizon (the number of steps) to a window where you think your model is still accurate.

Using models of dynamics to determine a policy is called *model predictive control*.

Week 4: Open-ended extensions

Much of the final week of your project will be taken up with writing up your final report. However, you are encouraged (especially if you make quick progress reaching this point) to extend your project into more complicated territory. There are two suggestions of potential directions below, but you are welcome to ‘just try out’ any ideas or questions that have sparked your interest over the last three weeks.

Reports will *not* be penalised for attempting extensions that do not work out! As long as you accurately describe your line of investigation (and explain/discuss *why* you think an idea ended up not working), you are doing good engineering.

Suggestion 1: Sensitivity and stability

Modify the problem to introduce noise and observe its effect.

Hint: start by introducing noise into the observed dynamics (but not in the real dynamics of the system). Refit the models and characterise the degradation in the prediction accuracy, then investigate what happens when you optimise and use the linear control policy (comparing it to the noise-free case). Then introduce noise in the actual dynamics of the system, repeat, and compare.

Suggestion 2: Nonlinear controllers

Attempt to improve control of the system by using a policy $p(X)$ that is nonlinear.

Hint: you can approach this similarly to the nonlinear model of the dynamics by using basis functions, or you might consider other nonlinear transformations of X .